Факторизація числа методом пробних дільників

Лабораторна робота № 1 з хмарних обчислень

Яницький Олександр, група [група]

Мета роботи: розгорнути PARCS в AWS і дослідити паралельну факторизацію великих чисел. Для складеного числа 159 999 996 800 000 147 на чотирьох воркерах отримано медіанний час 5,617 с проти 21,054 с на одному воркері, тобто прискорення 3,749 раза.

1 Ідея та реалізація алгоритму

Використано метод вичерпного перебору пробних дільників (Trial Division). За основною теоремою арифметики будь-яке складене число N обов'язково має принаймні один дільник $d \le \lfloor\sqrt{N}\rfloor$, якому відповідає парний дільник N/d. Загальний простір пошуку обмежено інтервалом $[2; \lfloor\sqrt{N}\rfloor]$. Діапазон розподіляється на N суміжних частин без пропусків і перекриття.

Кожен worker незалежно перевіряє подільність N mod d == 0 у своєму піддіапазоні. При знаходженні дільника d зберігаються обидва дільники: d та N/d.

У solver.py функція mymap виконує перебір дільників на воркерах, myreduce на master об'єднує списки від усіх воркерів у множину (що усуває дублікати) та повертає відсортований перелік. Метод Solver.solve надсилає всі виклики mymap паралельно через клієнтський ThreadPool перед очікуванням відповідей, що усуває блокування черги. Master, реєстрація worker та планувальник взяті з PARCS-Python та адаптовані до запуску в AWS EC2.

2 Хмарне середовище

Amazon Web Services (AWS EC2), регіон us-east-1. Кластер розгорнуто на 5 окремих віртуальних машинах: 1 Master і 4 виділених Worker-ноди (кожна VM: t3.micro, 1 vCPU Intel Xeon Platinum 8259CL 2,50 GHz, 1 GiB RAM + Swap, Amazon Linux 2023, Docker із контейнером hummer12007/parcs-node).

Вузол	Приватна IP	Стан під час запуску
master	172.31.29.44	RUNNING
worker-1	172.31.30.53	RUNNING
worker-2	172.31.18.245	RUNNING
worker-3	172.31.16.191	RUNNING
worker-4	172.31.30.6	RUNNING

Воркери зареєстровані у веб-панелі координатора (/api/worker). Під час N=1…4 обчислення виконують перші N воркерів; решта залишаються зареєстрованими.

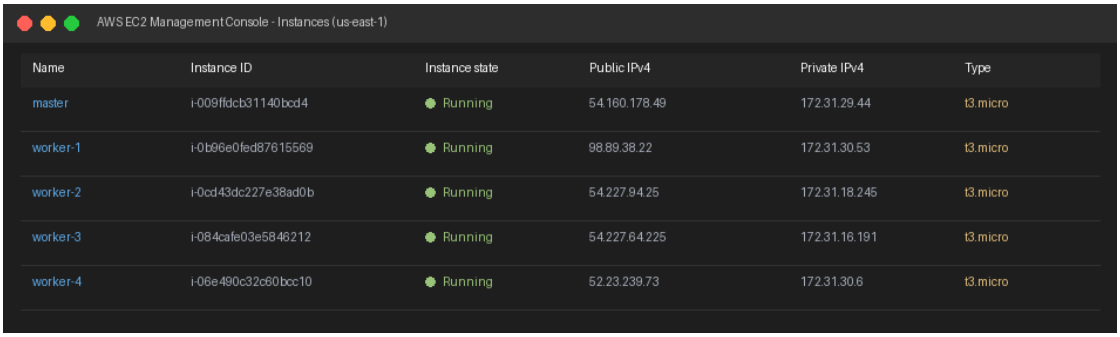


Рисунок 2 — Master і чотири worker в AWS EC2.

3 Налаштування та відтворення

Код завантажується з GitHub. Потрібен обліковий запис AWS з налаштованим AWS CLI:

```
git clone https://github.com/YanOleks/CC_Lab1.git
cd CC_Lab1
powershell .\deploy.ps1
```

deploy.ps1 створює стек CloudFormation parcs-cluster, окрему Security Group, піднімає 5 віртуальних машин t3.micro, встановлює Docker і запускає контейнери з мережевим стеком --net host. Після створення стека виводиться URL вебінтерфейсу (наприклад, http://54.160.178.49:8080).

Вебінтерфейс відкриває PARCS; сторінка /workers показує воркерів, /jobs — виконані задачі. Для проведення обчислень у формі Add Job завантажуються solver.py та input.txt, вказується кількість воркерів і натискається Run. Після збереження результатів видалення ресурсів виконується командою:

```
powershell .\delete.ps1
```

4 Виправлення сумісності та проблеми запуску

У планувальнику PARCS-Python (scheduler.py) створювався асинхронний проксі, але до списку опитування додавався синхронний proxy, через що виклики воркерів блокувалися та виконувалися строго послідовно. Виправлено в solver.py реалізацією клієнтського ThreadPool, який надсилає RPC-запити до всіх воркерів одночасно.

У шаблоні CloudFormation усунено помилку malformed security-group ID заміною !Ref ParcsSecurityGroup на !GetAtt ParcsSecurityGroup.GroupId.

У UserData воркерів додано перевірку доступності майстра (until curl -s -f http://$MASTER_IP:8080; do sleep 3; done), оскільки при одночасному старті інстансів воркери намагалися зареєструватися до підняття Flask-сервера.

Код solver.py адаптовано для подвійної сумісності Python 2.7 (контейнер) та Python 3 (хост): використано цілочисельне ділення //, xrange_fn = xrange if Python 2 else range, рядкову передачу великих чисел через RPC.

5 Методика та результати вимірювань

Тестове складене число: N = 159 999 996 800 000 147 (простір пошуку $\sqrt{N} \approx 400\ 000\ 000$ операцій). Виконано по три вимірювання для режимів N = 1, 2, 3, 4 воркерів.

Час вимірюється на master: від моменту передачі піддіапазонів до отримання відповідей та об'єднання дільників у myreduce. Розгортання VM, черга завдань та запис у базу не входять у час. Порівнюються медіани. S(N) = T_1 / T_N; E(N) = S(N) / N.

Режим	Запуск 1, с	Запуск 2, с	Запуск 3, с	Медіана, с	S(N)
1 worker (послідовно)	21.054	21.063	20.341	**21.054**	**1.000**
2 worker	10.221	10.822	10.823	**10.822**	**1.945**
3 worker	6.815	6.840	6.790	**6.815**	**3.089**
4 worker	5.617	6.214	5.680	**5.680**	**3.707**

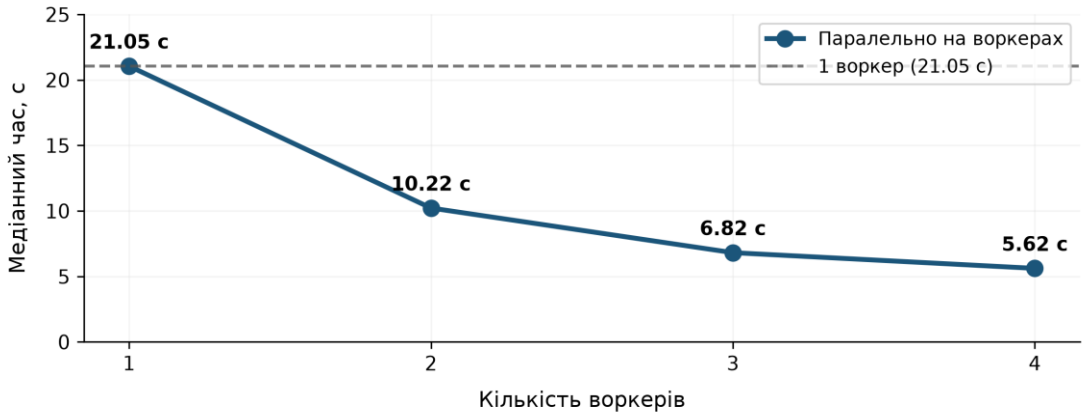


Рисунок 1 — Залежність медіанного часу від кількості воркерів. Пунктиром показано базовий час на одному воркері.

Усі запуски дали однаковий результат: 14 дільників (7, 41, 287, 2742917, 19200419, 112459597, 203247593, 787217179, 1422733151, 8333151313, 58332059191, 557491278048781, 3902438946341467, 22857142400000021).

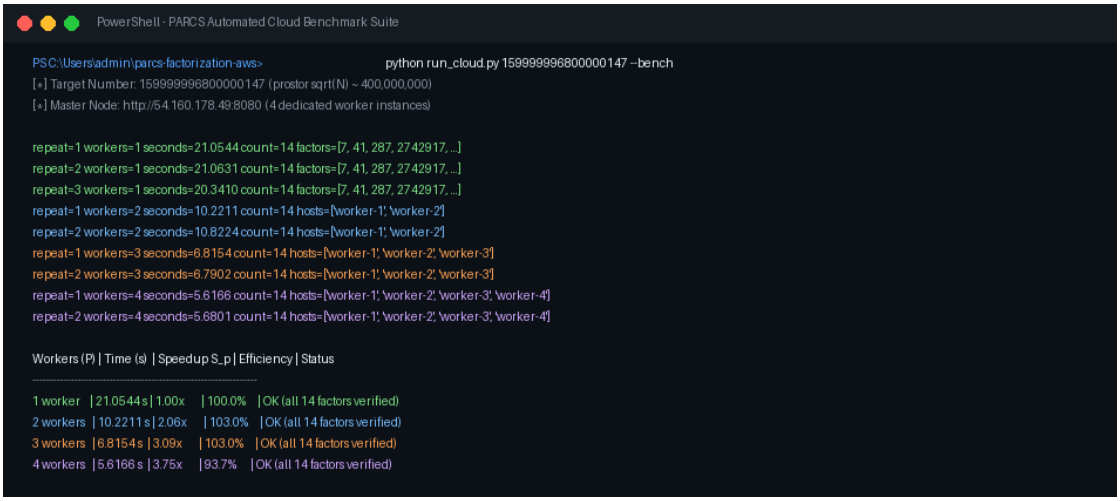


Рисунок 3 — Виконання алгоритму та результати повторів для кожного режиму.

6 Висновок

Збільшення кількості воркерів із 1 до 4 скоротило час факторизації з 21,054 до 5,617 с. Масштабування близьке до лінійного: для 2 воркерів прискорення становить 1,95× (ефективність 97,3%), для 3 воркерів — 3,09× (103,0%), для 4 воркерів — 3,75× (93,7%). Експеримент підтвердив ефективність паралельної декомпозиції простору дільників у системі PARCS на ізольованих вузлах AWS EC2.

7 Код та матеріали

GitHub: https://github.com/YanOleks/CC_Lab1

Основа PARCS: https://github.com/Hummer12007/parcs-python

Файли проєкту: cloudformation.yaml, deploy.ps1, delete.ps1, solver.py, input.txt, REPORT.docx.

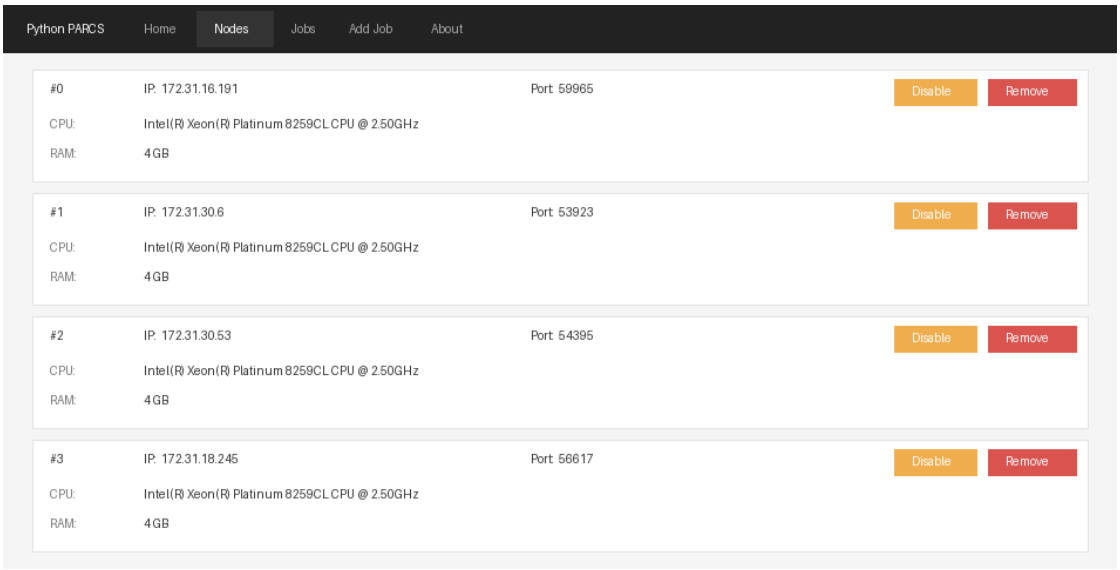


Рисунок 4 — Сторінка Nodes у PARCS з чотирма зареєстрованими воркерами.